```
#include <iostream>
#include <climits>
using namespace std;

int main()
{
   int n;
   int graph[10][10];
   int parent[10];

   cout << "Enter number of vertices: ";
   cin >> n;

   cout << "Enter cost matrix:\n";
   cout << "Enter 0 if there is no edge.\n";

   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         cin >> graph[i][j];
      }

      parent[i] = -1;
   }

   int edges = 0;
   int totalCost = 0;

   cout << "\nEdges in Minimum Spanning Tree:\n";

   while (edges < n - 1)
   {
      int minCost = INT_MAX;
      int u = -1;
      int v = -1;

      // Find the minimum cost edge
      for (int i = 0; i < n; i++)
      {
         for (int j = i + 1; j < n; j++)
         {
            if (graph[i][j] != 0 && graph[i][j] < minCost)
            {
               minCost = graph[i][j];
               u = i;
               v = j;
            }
```

```
        }
      }

      // Find parent of u
      int x = u;
      while (parent[x] != -1)
      {
        x = parent[x];
      }

      // Find parent of v
      int y = v;
      while (parent[y] != -1)
      {
        y = parent[y];
      }

      // If they don't form a cycle
      if (x != y)
      {
        cout << u << " - " << v
           << "  Cost: " << minCost << endl;

        totalCost += minCost;
        parent[x] = y;
        edges++;
      }

      // Remove this edge
      graph[u][v] = 0;
      graph[v][u] = 0;
    }

    cout << "\nMinimum Cost of Spanning Tree = "
       << totalCost << endl;

    return 0;
}
```

Enter number of vertices: 3

Enter cost matrix:

Enter 0 if there is no edge.

0

1

2

4

2

3

1

2

4

Edges in Minimum Spanning Tree:

0 - 1 Cost: 1

0 - 2 Cost: 2

Minimum Cost of Spanning Tree = 3

3